

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées Al Hoceima

COMPTE RENDU

Hyper-paramètres des réseaux de neurones

Pr. Mohamed CHERRADI

Chaimae OUHADI

Filière : TDIA 2 - S4

1. Introduction

La qualité d'un modèle dans les réseaux de neurones ne se détermine pas seulement par l'algorithme choisi ou les données utilisées, mais aussi par le choix des **hyper-paramètres**. Ces derniers, contrairement aux **paramètres** du modèle (biais et poids) qui permettent au modèle d'apprendre les règles à partir des données et changent par l'algorithme à chaque itération, doivent être définis avant le début de l'entraînement et restent fixes. Ils contrôlent le fonctionnement du modèle et déterminent la mise à jour des poids, la complexité de l'architecture du réseau ou encore l'intensité de la régularisation appliquée pour éviter le surapprentissage.

Les **hyper-paramètres** sont des variables de configuration que les data scientists définissent à l'avance pour gérer le processus d'entraînement d'un modèle de machine learning. Trouver la bonne combinaison d'hyper-paramètres est essentiel pour optimiser la performance des modèles d'apprentissage supervisé et d'apprentissage non supervisé.

Dans les réseaux de neurones, on distingue généralement deux types d'hyper-paramètres: les **hyper-paramètres d'optimisation** qui contrôlent le processus d'apprentissage: tels que les méthodes d'apprentissage à partir des données et d'optimisation, la pas d'apprentissage, la taille de batch,... etc; Et les **hyper-paramètres d'architecture** qui définissent la structure même du réseau, sa capacité à représenter des fonctions complexes, et directement ses performances: le nombre de couches, le nombre de neurones par couches, fonctions d'activation,... etc.

Le réglage approprié de ces hyper-paramètres est crucial, car il permet d'améliorer la performance globale du modèle, il influence directement la vitesse de convergence, la stabilité de l'entraînement ainsi que la capacité du modèle à bien généraliser sur de nouvelles données. C'est pourquoi la compréhension et la maîtrise de ces hyper-paramètres constituent une étape essentielle dans la conception de modèles de deep learning performants.

2. Hyper-paramètres d'Optimisation

Les **hyper-paramètres d'optimisation** contrôlent la manière dont le réseau de neurones apprend à partir des données, c'est-à-dire la façon dont les poids sont mis à jour pour minimiser la fonction de perte. Un mauvais réglage peut rendre l'apprentissage très lent, instable ou conduire à un modèle qui ne généralise pas bien, même si l'architecture est correcte.

2.1. Les méthodes d'optimisation (Optimizers)

Les méthodes d'optimisation (ou « **optimizer** ») définissent la stratégie utilisée pour mettre à jour les poids à partir du gradient de la perte. Il existe plusieurs types d'optimizers tels que: **SGD** (Stochastic Gradient Descent), **Momentum** (qu'on peut traduire par "élan" qui est un mouvement physique ou figuré impétueux), **RMSprop** (Root Mean Square Propagation), **Adam** (Adaptive Moment Estimation),...

Le choix de l'optimizer a un impact sur la vitesse de convergence, la stabilité de l'entraînement et parfois sur la qualité du minimum trouvé.

2.1.1. SGD: Stochastic Gradient Descent

Le SGD, ou descente de gradient stochastique, est la version la plus simple et la plus ancienne des optimiseurs. Il met à jour les poids à partir des gradients calculés sur un seul exemple (ou un mini-batch), ce qui rend l'apprentissage plus rapide que la descente de gradient classique calculée sur tout le dataset.

Avantages: simple à comprendre, faible coût mémoire, convergent vers de bonnes solutions.

Inconvénients: mises à jour bruitées, convergence lente, très sensible au choix du learning rate.

2.1.2. Momentum

Le Momentum améliore le SGD en ajoutant une sorte d'inertie aux mises à jour. L'idée vient de la physique : une bille qui roule accumule de la vitesse dans une direction.

Si le gradient pointe vers la même direction plusieurs itérations de suite, on accélère dans cette direction. Si le gradient oscille, le momentum les amortit. Et on obtient donc une convergence plus rapide et plus stable.

Le SGD avec Momentum ajoute donc une mémoire des gradients passés, ce qui permet de lisser les oscillations et d'accélérer la convergence dans la direction la plus prometteuse.

Avantages: convergence plus rapide, moins d'oscillations, passe plus facilement les plateaux.

Inconvénients: ajoute un hyper-paramètre β à régler. Peut "dépasser" le minimum (overshooting).

2.1.3. RMSprop: Root Mean Square Propagation

Le **RMSprop** est un algorithme d'optimisation adaptatif majeur dans l'apprentissage profond proposé par **Geoffrey Hinton** en **2012** lors de son cours en ligne sur Coursera "**Neural Networks for Machine Learning**". Il **accélère la descente de gradient** et il est utilisé pour l'entraînement des modèles à base de réseaux de neurones profonds. Son idée consiste à **adapter le taux d'apprentissage** pour chaque paramètre en fonction de l'historique récent des gradients.

Si un paramètre a des gradients **grands et fréquents**, on **réduit son learning rate**. S'il a des gradients **petits et rares**, on **l'augmente**. Chaque paramètre a ainsi son propre pas d'apprentissage, adapté à sa dynamique.

Avantages: très efficace sur des fonctions de coût non-stationnaires, particulièrement adapté aux RNN et aux problèmes avec des gradients très variables.

Inconvénients: ne bénéficie pas du Momentum, peut converger vers des solutions sous-optimales.

2.1.4. Adam: Adaptive Moment Estimation

Adam, proposé par **Kingma & Ba** en **2014**, est aujourd'hui l'optimiseur le plus utilisé en Deep Learning. Son idée de génie est de **combiner le Momentum** (pour la vitesse) et **RMSprop** (pour l'adaptation par paramètre) dans un seul algorithme.

Adam maintient deux estimations statistiques des gradients :

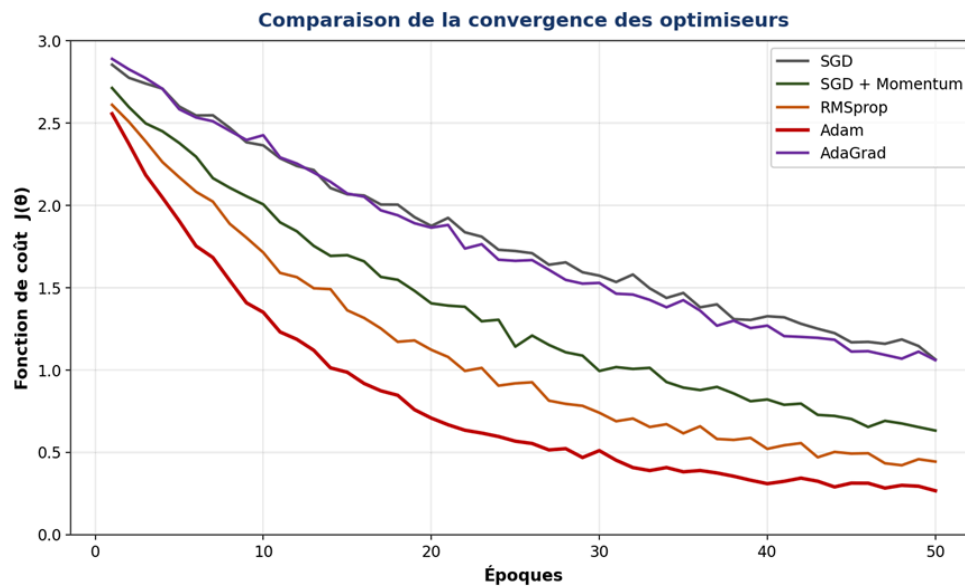
- **Le premier moment (moyenne):** comme dans Momentum, il accumule les gradients récents.
- **Le second moment (variance):** comme dans RMSprop, il accumule les carrés des gradients récents.

Valeurs recommandées par les auteurs (à utiliser par défaut) :

- $\alpha = 0.001$ (learning rate)
- $\beta_1 = 0.9$ (momentum du 1er moment)
- $\beta_2 = 0.999$ (momentum du 2ème moment)
- $\epsilon = 1e-8$ (stabilité numérique)

Avantages: convergence très rapide, fonctionne bien sans réglage fin, adaptatif par paramètre, excellent pour les gradients épars (NLP).

Inconvénients: consomme plus de mémoire (2 vecteurs de moments en plus), généralise parfois un peu moins bien que SGD+Momentum sur certains problèmes de vision.



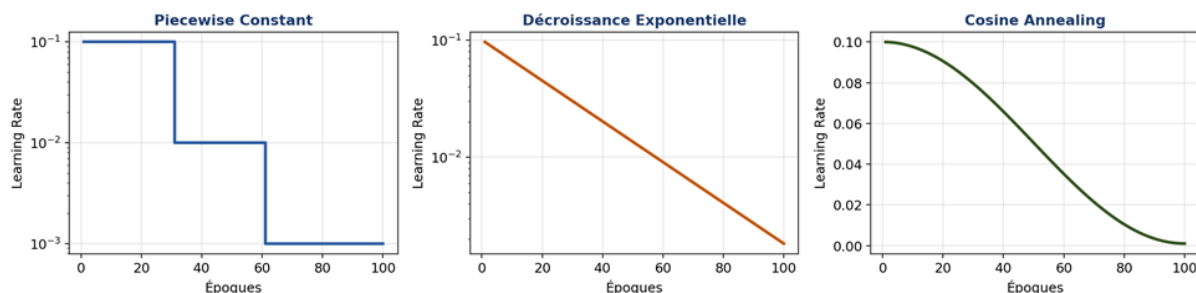
2.2. Le pas d'apprentissage

Le **pas d'apprentissage** est l'un des hyper-paramètres les plus critiques. Il détermine l'ampleur de la mise à jour des poids à chaque itération de l'algorithme de descente de gradient.

- S'il est trop grand, les mises à jour sont brutales et le modèle peut « sauter » autour du minimum sans jamais converger, voire diverger complètement.
- S'il est trop petit, l'apprentissage devient très lent et peut se bloquer dans des minima locaux ou des plateaux de la fonction de perte.

Parmi les stratégies de planification de learning rate, on trouve: **Piecewise Constant Schedule**, **Décroissance Exponentielle** et **Planification en Puissance (Polynomial Decay)**.

Stratégies de planification du Learning Rate (Learning Rate Schedulers)



Piecewise Constant Schedule: on fixe un learning rate élevé au début pour accélérer l'apprentissage, puis on le divise par un facteur après un certain nombre d'époques, et ainsi de suite.

Avantages: simple, très utilisé dans les architectures célèbres (ResNet, VGG).

Inconvénients: nécessite de prédéfinir les époques de changement, pas de réelle adaptation.

Décroissance Exponentielle: le learning rate diminue de manière continue et exponentielle au fil des époques: $\alpha_t = \alpha_0 \cdot e^{(-k \cdot t)}$, Où α_0 est le learning rate initial, k est le taux de décroissance, et t est l'époque courante.

Planification en Puissance (Polynomial Decay): le learning rate diminue selon une fonction en puissance: $\alpha_t = \alpha_0 / (1 + k \cdot t)^p$

Lorsque $p = 1$, on obtient la décroissance inverse time decay. Cette stratégie offre un bon équilibre entre décroissance rapide au début et lente à la fin.

En pratique, on choisit un learning rate initial (par exemple 0.1, 0.01 ou 0.001 selon le problème) et on peut utiliser des techniques comme les schedulers ou les méthodes adaptatives (RMSProp, Adam) pour l'ajuster automatiquement au cours de l'entraînement.

2.3. Nombre d'itérations

Rappel:

- 1 époque = un passage complet sur tout le dataset.
- 1 itération = traitement d'un seul batch.
- Nombre d'itérations par époque = Taille totale du dataset / Taille du batch.

Comment choisir le nombre d'itérations (ou d'époques) ?

Il n'existe quelques principes qui guident le choix :

- Trop peu d'époques donnent l' underfitting. Le modèle n'a pas eu le temps d'apprendre les patterns des données.
- Trop d'époques donnent l' overfitting. C'est-à-dire que le modèle commence à mémoriser le bruit des données d'entraînement.
- L'idéal donc est d'utiliser un jeu de validation pour surveiller la performance pendant l'entraînement, et arrêter dès que celle-ci cesse de s'améliorer.

Early Stopping (arrêt anticipé) consiste à surveiller la perte sur le jeu de validation à **chaque époque**, et à **arrêter automatiquement l'entraînement** dès que cette perte cesse de diminuer (ou commence à augmenter).

Paramètres clés de l'Early Stopping :

- **Patience:** nombre d'époques sans amélioration avant d'arrêter. Une valeur entre 5 et 20 est généralement appropriée.
- **Restore best weights:** on restaure les poids correspondant à la meilleure performance observée, pas les derniers.
- **Métrique surveillée:** typiquement la val_loss (perte de validation) ou val_accuracy.

2.4. Régularisation

La régularisation désigne l'ensemble des techniques visant à **réduire le sur-apprentissage (overfitting)**, c'est-à-dire la tendance du modèle à mémoriser les données d'entraînement plutôt qu'à en extraire des patterns généralisables.

2.4.1. Régularisation L1 (Lasso)

La régularisation L1 **ajoute à la fonction de coût un terme de pénalité égal à la somme des valeurs absolues des coefficients du modèle** (poids).

$$J_{L1}(w) = J(w) + \lambda \cdot \sum |w_i|$$

avec λ : hyper-paramètre de régularisation (force de la pénalité)

Si λ trop grand: sous-apprentissage (underfitting), ET si λ trop petit: pas d'effet régularisant.

Cela encourage le modèle en réduisant certains coefficients à zéro, ce qui permet d'effectuer une sélection efficace des caractéristiques. Le modèle qui en résulte est plus simple et plus facile à interpréter, car il ne sélectionne que les caractéristiques les plus importantes tout en écartant celles qui le sont moins.

2.4.2. Régularisation L2 (Ridge / Weight Decay)

La régularisation L2 **ajoute à la fonction de coût un terme de pénalité égal au carré des amplitudes des coefficients du modèle**. Il tend à réduire les coefficients des caractéristiques moins importantes vers zéro, mais il est rare qu'il les ramène exactement à zéro. La régularisation L2 est efficace pour prévenir le surajustement en pénalisant les poids importants et, par conséquent, en réduisant la complexité du modèle.

$$J_{L2}(w) = J(w) + \lambda \cdot \sum w_i^2$$

$$\text{Avec } w_{i+1} = w_i \cdot (1 - \alpha \cdot \lambda) - \alpha \cdot \nabla J(w)$$

2.4.3. ElasticNet

ElasticNet **combine les deux régularisations L1 et L2** avec **deux hyper-paramètres λ_1 et λ_2** (ou alternativement α et le ratio de mélange ρ). Cela permet de bénéficier à la fois de la sparsité (L1) et de la stabilité numérique (L2).

$$J_{EN}(w) = J(w) + \lambda_1 \cdot \sum |w_i| + \lambda_2 \cdot \sum w_i^2$$

On utilise cette méthode lorsque le nombre de features est très élevé et qu'on suspecte que seule une partie est pertinente, mais sans certitude totale.

3. Hyper-paramètres d'architecture

Si les **hyper-paramètres d'optimisation** contrôlent comment le modèle apprend, les **hyper-paramètres d'architecture** définissent ce que le modèle est. Ils déterminent la structure même du réseau, sa capacité à représenter des fonctions complexes, et directement ses performances.

3.1. Nombre de Couches et Nombre de Neurones

3.1.1. Nombre de Couches (Profondeur du réseau)

La profondeur d'un réseau (depth) désigne **le nombre de couches cachées** entre la couche d'entrée et la couche de sortie. C'est un hyper-paramètre fondamental qui détermine la capacité du réseau à apprendre des représentations hiérarchiques.

- Réseau **peu profond** (1–2 couches) : adapté aux problèmes simples, linéairement séparables ou quasi-linéaires. Risque d'underfitting sur des problèmes complexes.
- Réseau **modérément profond** (3–10 couches) : bon compromis pour la plupart des tâches de classification/régression sur données tabulaires.
- Réseau **très profond** (10+ couches) : nécessaire pour la vision par ordinateur (CNN) et le traitement du langage naturel. Requiert des techniques spéciales (BatchNorm, ResNet connections, etc.) pour éviter la disparition du gradient.

Théorème d'approximation universelle: un réseau avec **une seule couche cachée** et un **nombre suffisant de neurones** peut théoriquement **approximer toute fonction continue**. En pratique, cependant, on préfère augmenter la profondeur plutôt que la largeur, car les réseaux profonds apprennent des représentations plus efficaces avec moins de paramètres.

3.1.2. Nombre de Neurones (Largeur du réseau)

La largeur (width) d'une couche désigne **le nombre de neurones qu'elle contient**. Plus une couche est large, plus elle peut apprendre de représentations complexes, mais au prix d'un coût computationnel accru et d'un risque d'overfitting.

- **Couche d'entrée** : toujours égale au nombre de features (variables d'entrée).
- **Couches cachées** : typiquement des puissances de 2 (32, 64, 128, 256, 512, 1024). Une architecture en entonnoir (décroissante) est souvent efficace.
- **Couche de sortie** : 1 neurone pour la régression ou la classification binaire, N neurones pour la classification multi-classes (avec Softmax).

3.2. Types de couches

3.2.1. Couche Fully Connected (FC) ou Couche Dense

Une **couche Fully Connected** (ou **Dense**) est la couche de base des réseaux de neurones. **Chaque neurone de cette couche est connecté à tous les neurones de la couche précédente**. C'est la brique fondamentale du Perceptron Multi-Couches (MLP).

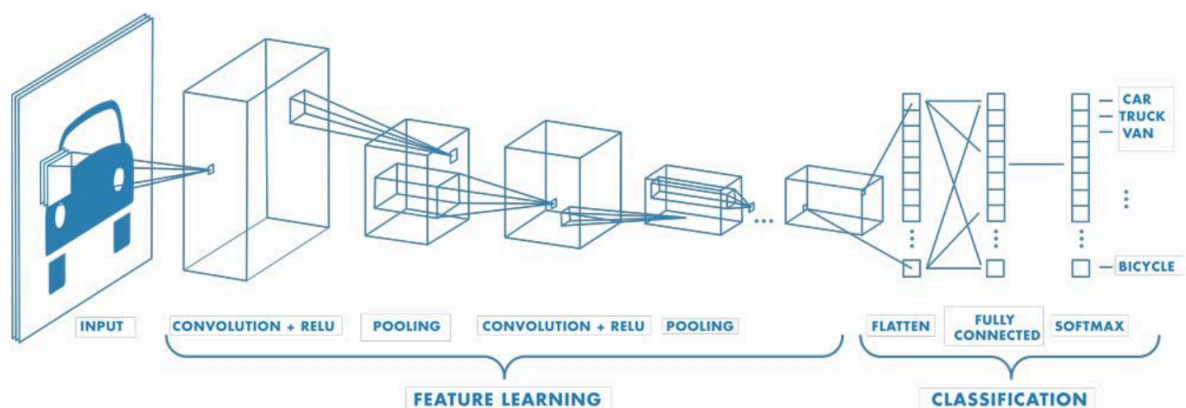
3.2.2. Convolutional Neural Network : CNN

Un **CNN** est un **réseau spécialisé dans le traitement de données à structure spatiale**, en particulier les images. Au lieu de connecter chaque pixel à chaque neurone, il utilise des **filtres (kernels)** qui glissent sur l'entrée pour extraire des features locales.

Un **CNN** comporte une couche d'entrée, une couche de sortie et de nombreuses couches intermédiaires cachées. Ces couches effectuent des opérations pour modifier les données, afin d'en apprendre les caractéristiques spécifiques. Les trois couches les plus courantes sont : la convolution, l'activation (ou ReLU) et le pooling.

- **La couche de convolution** applique un ensemble de filtres convolutifs aux images en entrée, chacun d'entre eux activant certaines caractéristiques des images.
- **La couche ReLU (Rectified linear unit)** favorise un apprentissage plus rapide et plus efficace en remplaçant les valeurs négatives par des zéros et en conservant les valeurs positives. Ce procédé est parfois appelé activation, car seules les caractéristiques activées sont transmises à la couche suivante.
- **La couche de pooling** simplifie la sortie en réalisant un sous-échantillonnage non linéaire, ce qui permet de réduire le nombre de paramètres que le réseau doit apprendre.

Ces opérations sont **répétées sur des dizaines ou des centaines de couches**, chaque couche apprenant à identifier différentes caractéristiques.

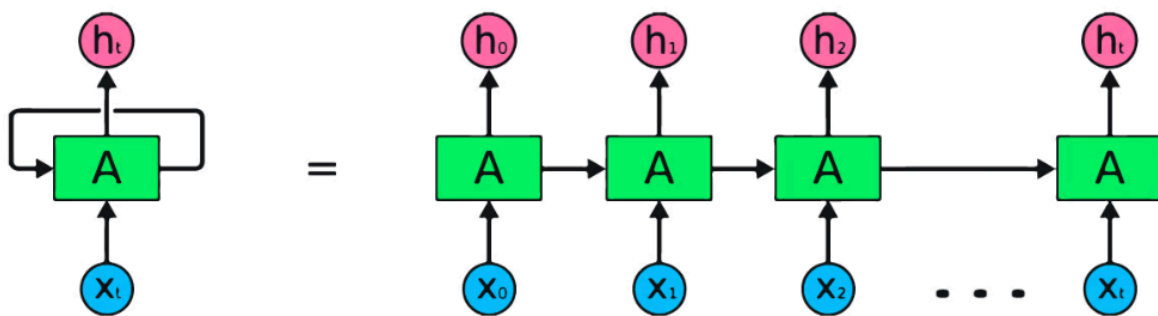


Exemple de réseau comportant de nombreuses couches convolutives. Des filtres sont appliqués à chaque image du jeu d'apprentissage avec différentes résolutions, puis la sortie de chaque image convoluée est utilisée comme entrée de la couche suivante.

3.2.3. Réseau de Neurones Récurent: RNN

Un **RNN** est conçu pour **traiter des données séquentielles** (texte, audio, séries temporelles). Il possède **des connexions récurrentes** qui lui permettent de maintenir une **mémoire interne** de l'état précédent, propagée à l'étape suivante.

Le schéma ci-dessous est une représentation simple des réseaux neuronaux récurrents. Si nous prévoyons les cours des actions en utilisant des données simples [45,56,45,49,50,...], chaque entrée de X_0 à X_t contiendra une valeur passée. Par exemple, X_0 aura 45, X_1 aura 56, et ces valeurs sont utilisées pour prédire le nombre suivant dans une séquence.



3.3. Fonctions d'Activation

Les **fonctions d'activation** font partie intégrante des réseaux neuronaux et leur permettent d'apprendre **des schémas complexes** dans les données. Ils transforment le signal d'entrée d'un nœud d'un réseau neuronal en un signal de sortie qui est ensuite transmis à la couche suivante. Sans fonctions d'activation, les réseaux neuronaux seraient limités à la modélisation de relations linéaires entre les entrées et les sorties.

Les fonctions d'activation introduisent des **non-linéarités**, ce qui permet aux réseaux neuronaux d'**apprendre des correspondances très complexes entre les entrées et les sorties**.

Le choix de la bonne fonction d'activation est crucial pour former des réseaux neuronaux qui généralisent bien et fournissent des prédictions précises.

3.3.1. Sigmoid (Logistique)

La fonction d'activation **sigmoïde**, souvent représentée par $\sigma(x)$, est une fonction lisse et continuellement différentiable, historiquement importante dans le développement des réseaux neuronaux. La fonction d'activation sigmoïde a la forme mathématique suivante :

$$f(x) = 1 / (1 + e^{-x})$$

Il prend une valeur réelle en entrée et la réduit à **une valeur comprise entre 0 et 1**. La fonction sigmoïde a une courbe en forme de "S" qui asymptote à 0 pour les grands nombres négatifs et à 1 pour les grands nombres positifs. Les résultats peuvent être facilement interprétés comme des probabilités, ce qui le rend naturel pour les problèmes de classification binaire.

Les unités sigmoïdes étaient populaires dans les premiers réseaux neuronaux car le gradient est le plus fort lorsque la sortie de l'unité est proche de 0,5, ce qui permet un apprentissage efficace par rétropropagation. Cependant, les unités sigmoïdes souffrent du problème du "gradient disparaissant" qui entrave l'apprentissage dans les réseaux neuronaux profonds.

3.3.2. Tanh (Tangente Hyperbolique)

La fonction d'activation **tanh (tangente hyperbolique)** est définie comme suit :

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

La fonction tanh produit des **valeurs comprises entre -1 et +1**. Cela signifie qu'elle peut traiter les valeurs négatives plus efficacement que la fonction sigmoïde, dont la plage est comprise entre 0 et 1.

Contrairement à la fonction sigmoïde, tanh est centrée sur le zéro, ce qui signifie que sa sortie est symétrique par rapport à l'origine du système de coordonnées. Ceci est souvent considéré comme un avantage car cela peut aider l'algorithme d'apprentissage à converger plus rapidement.

Comme la sortie de tanh se situe entre -1 et +1, ses gradients sont plus forts que ceux de la fonction sigmoïde. Des gradients plus forts permettent souvent un apprentissage et une convergence plus rapides pendant l'apprentissage, car ils ont tendance à mieux résister au problème des gradients qui s'évanouissent, par rapport aux gradients de la fonction sigmoïde.

Malgré ces avantages, la fonction tanh souffre toujours du problème de la disparition du gradient. Au cours de la rétropropagation, les gradients de la fonction tanh peuvent devenir très faibles (proches de zéro). Cette question est particulièrement problématique pour les réseaux profonds comportant de nombreuses couches ; les gradients de la fonction de perte peuvent devenir trop faibles pour apporter des changements significatifs aux poids pendant la formation, car ils se propagent vers les couches initiales. Cela peut ralentir considérablement le processus d'apprentissage et entraîner de mauvaises propriétés de convergence.

La fonction tanh est fréquemment utilisée dans les couches cachées d'un réseau neuronal. En raison de sa nature centrée sur le zéro, lorsque les données sont également normalisées pour avoir une moyenne nulle, la formation peut être plus efficace.

3.3.3 ReLU (Rectified Linear Unit)

La fonction d'activation de l'unité linéaire rectifiée (ReLU) a la forme suivante :

$$f(x) = \max(0, x)$$

Le seuil d'entrée est fixé à zéro, ce qui renvoie 0 pour les valeurs négatives et l'entrée elle-même pour les valeurs positives.

Pour les entrées supérieures à 0, la ReLU agit comme une fonction linéaire avec un gradient de 1. Cela signifie qu'il ne modifie pas l'échelle des entrées positives et permet au gradient de passer sans changement pendant la rétropropagation. Cette propriété est essentielle pour atténuer le problème du gradient de fuite.

Même si la ReLU est linéaire pour la moitié de son espace d'entrée, il s'agit techniquement d'une fonction non linéaire car elle possède un point non différentiable à $x=0$, où elle change brusquement de x . Cette non-linéarité permet aux réseaux neuronaux d'apprendre des modèles complexes

Étant donné que la ReLU produit un résultat nul pour toutes les entrées négatives, elle conduit naturellement à des activations éparpillées ; à tout moment, seul un sous-ensemble de neurones est activé, ce qui permet d'améliorer l'efficacité des calculs.

La fonction ReLU est peu coûteuse en termes de calcul car elle implique un simple seuillage à zéro. Cela permet aux réseaux de s'étendre à de nombreuses couches sans augmentation significative de la charge de calcul, par rapport à des fonctions plus complexes telles que tanh ou sigmoïde.

3.3.4. Softmax

La **fonction d'activation softmax**, également connue sous le nom de fonction **exponentielle normalisée**, est particulièrement utile dans le contexte des problèmes de **classification multi-classes**. Cette fonction opère sur un vecteur, souvent appelé **logit**, qui représente les **prédictions brutes** ou les **scores pour chaque classe** calculés par les couches précédentes d'un réseau neuronal.

Pour un vecteur d'entrée x avec des éléments $x_1, x_2, \dots, x_C$, la fonction softmax est définie comme suit :

$$f(x_i) = e^{x_i} / \sum_j e^{x_j}$$

La sortie de la fonction softmax est une **distribution de probabilité** dont la **somme est égale à un**. Chaque élément de la sortie représente la probabilité que l'entrée appartienne à une classe particulière.

L'utilisation de la fonction exponentielle garantit que toutes les valeurs de sortie sont non négatives. Ce point est crucial car les probabilités ne peuvent pas être négatives.

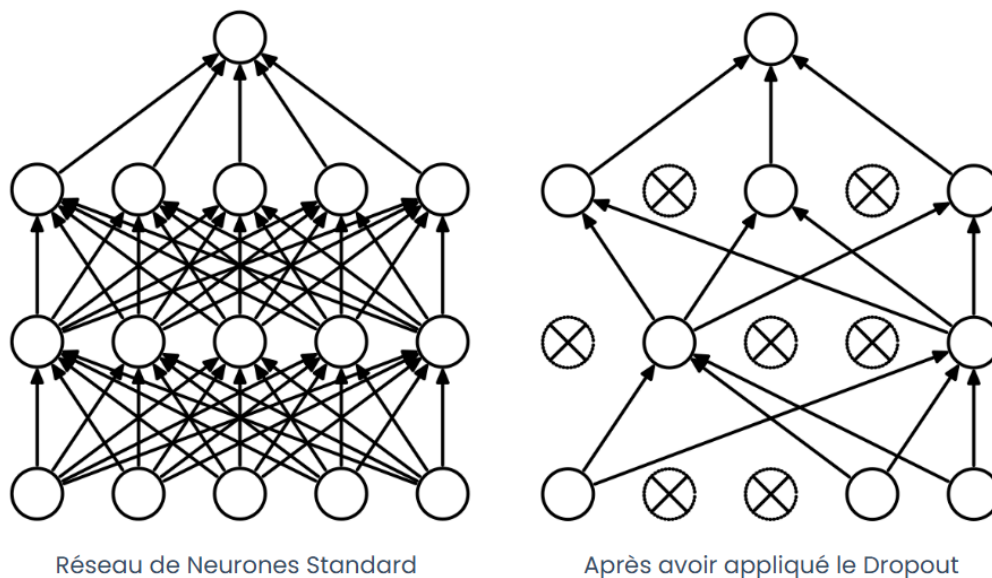
Softmax amplifie les différences dans le vecteur d'entrée. Même de petites différences dans les valeurs d'entrée peuvent entraîner des différences substantielles dans les probabilités de sortie, la ou les valeurs d'entrée les plus élevées ayant tendance à dominer dans la distribution de probabilités résultante.

Le Softmax est généralement utilisé dans la couche de sortie d'un réseau neuronal lorsque la tâche consiste à classer une entrée dans l'une de plusieurs (plus de deux) catégories possibles (classification multi-classes).

3.4. Dropout

L'overfitting est un problème fréquent lors de l'entraînement d'un modèle de Deep Learning, mais une technique existe pour le contrer : **le Dropout**.

Cette technique, expliquée dans un papier de recherche par **Srivastava et al.** en **2014**, permet de réduire l'overfitting lors de l'entraînement du modèle. Le terme "Dropout" fait référence à la suppression de neurones dans les couches d'un modèle de Deep Learning. En fait, **on désactive temporairement certains neurones dans le réseau, ainsi que toutes ses connexions entrantes et sortantes** :



Le choix des neurones à désactiver est **aléatoire**. On attribue une probabilité **p** à tous les neurones qui détermine leur activation. Lorsque $p = 0.1$, chaque neurone a une chance sur 10 d'être désactivé.

À chaque epoch, on applique cette désactivation aléatoire. C'est-à-dire qu'à chaque passe (forward propagation) le modèle apprendra avec une configuration de neurones différentes, les neurones s'activant et se désactivant aléatoirement. Cette procédure génère effectivement des modèles légèrement différents avec des configurations de neurones différentes à chaque itération.

L'idée est de perturber les caractéristiques apprises par le modèle. Habituellement, l'apprentissage du modèle repose sur la synchronicité des neurones... avec le Dropout, le modèle doit exploiter chaque neurone individuellement, ses voisins pouvant être désactivés aléatoirement à tout instant.

Sur Keras & Tensorflow:

Il suffit d'ajouter une couche Dropout et d'indiquer la probabilité de désactivation souhaitée. La probabilité par défaut est de 0.5, dans cet exemple on prend 0.2 :

```
tf.keras.layers.Dropout(0.2)
```

Sur PyTorch:

```
torch.nn.Dropout(p=0.2)
```

3.5. Paramètres Spécifiques aux CNN

3.5.1. Filtres (Kernels) : Type et Taille

Un **filtre (ou kernel)** est **une petite matrice de poids** apprise par le réseau, qui est convoluée avec l'entrée pour produire une feature map. Chaque filtre est sensible à un pattern particulier (bord, coin, texture...).

- **Taille du filtre** : Typiquement 1×1 , 3×3 , 5×5 , 7×7 . Les filtres 3×3 sont les plus utilisés (standard dans VGG, ResNet). Les filtres 1×1 sont utilisés pour réduire les dimensions (bottleneck).
- **Nombre de filtres** : Détermine le nombre de feature maps produites. Typiquement 32, 64, 128, 256, 512 filtres par couche, augmentant au fil des couches.

Calcul de la dimension de la sortie:

$$n_{out} = \text{floor}((n_{in} + 2 \cdot P - F) / S) + 1$$

n_{in} = taille entrée, F = taille filtre, P = padding, S = stride

5.3.2. Stride

Le **stride (pas de déplacement)** définit de **combien de pixels le filtre se déplace à chaque étape lors de la convolution**. C'est un hyper-paramètre qui contrôle la résolution de la feature map de sortie.

- **Stride=1**: Le filtre se déplace d'un pixel à la fois. La feature map a une taille proche de l'entrée (dépend du padding). C'est la valeur par défaut.
- **Stride=2**: Le filtre saute un pixel. La feature map est deux fois plus petite. Utilisé à la place du max-pooling dans certaines architectures modernes.
- **Stride > 2**: Sous-échantillonnage agressif. Utilisé en début de réseau pour réduire rapidement la dimension spatiale (ex: première couche de ResNet : conv 7×7 , stride=2).

5.3.3 Padding

Le **padding** consiste à **ajouter des pixels supplémentaires (généralement des zéros) autour des bords de l'entrée** avant d'appliquer le filtre. Il permet de **contrôler la dimension de la feature map** de sortie et d'éviter la perte d'information aux bords.

- **Padding 'valid' (sans padding)**: Aucun padding ajouté. La feature map est plus petite que l'entrée. Formule : $n_{out} = (n_{in} - F) / S + 1$
- **Padding 'same' (zero-padding)**: Du padding est ajouté pour que la feature map ait exactement la même dimension spatiale que l'entrée (pour $S=1$). Formule : $P = (F-1)/2$.

3.6. Couches de pooling

Le **pooling** est une opération de **sous-échantillonnage** qui **réduit la dimension spatiale des feature maps**, ce qui permet de diminuer le nombre de paramètres, d'accélérer le calcul et d'introduire une certaine invariance à la translation.

3.6.1. Max Pooling

Le **Max Pooling** sélectionne la **valeur maximale** dans chaque fenêtre de pooling. C'est l'opération de pooling la plus utilisée dans les **CNN**.

Il permet de détecter si une feature est présente quelque part dans la région, robuste aux petites translations. Utilisé dans VGG, AlexNet, LeNet.

3.6.2. Average Pooling

L'**Average Pooling** calcule la **moyenne** de toutes les valeurs dans la fenêtre de pooling. Il conserve l'information globale de la région, lisse les activations. Moins agressif que le Max

Pooling. Or il est moins courant que Max Pooling dans les couches intermédiaires, mais souvent utilisé comme Global Average Pooling en fin de réseau.

3.6.3. Global Pooling (Global Average/Max Pooling)

Le **Global Pooling** applique l'opération de pooling sur *toute la feature map (pas seulement une fenêtre)*. Il réduit chaque feature map à un seul scalaire.

- **Global Average Pooling (GAP):** Calcule la moyenne de toute la feature map. Remplace la couche Flatten + FC dans les architectures modernes (ResNet, MobileNet). Avantage : beaucoup moins de paramètres.
- **Global Max Pooling (GMP):** Retient la valeur maximale de chaque feature map.

3.7. Transfer Learning et Architectures Spécifiques

3.7.1. Concept du Transfer Learning

Le Transfer Learning (apprentissage par transfert) est une technique qui consiste à réutiliser un modèle pré-entraîné sur une large base de données (comme ImageNet avec 1.2 million d'images et 1000 classes) pour résoudre une tâche différente, souvent avec peu de données. Les couches profondes d'un CNN apprennent des features génériques : bords (couches basses), textures et motifs (couches moyennes), parties d'objets (couches hautes). Ces features sont utiles pour de nombreuses tâches visuelles.

3.7.2. VGG (Visual Geometry Group)

C'est une architecture développée par l'équipe VGG d'Oxford (Simonyan & Zisserman, 2014). A remporté ILSVRC 2014 (ImageNet Challenge). Le VGG est remarquable par sa simplicité et son architecture uniforme : elle utilise exclusivement des filtres 3×3 avec $\text{stride}=1$ et $\text{padding}=1$, suivis de couches de Max Pooling 2×2 .

- VGG-16 : 13 couches convolutionnelles + 3 couches FC. ~138 millions de paramètres.
- VGG-19 : 16 couches convolutionnelles + 3 couches FC. ~144 millions de paramètres.

3.7.3. ResNet (Residual Network)

Introduit par He et al. (Microsoft Research, 2015). Vainqueur de ILSVRC 2015. Représente une révolution dans le design des réseaux profonds. Les connexions résiduelles (Skip Connections). Au lieu d'apprendre une transformation $H(x)$, les couches apprennent le résidu $F(x) = H(x) - x$. La sortie devient : $H(x) = F(x) + x$.

4. Conclusion

Ce document présente une vue d'ensemble exhaustive des hyper-paramètres des réseaux de neurones, organisés selon deux grandes catégories : les hyper-paramètres d'optimisation et les hyper-paramètres d'architecture.

La maîtrise de ces hyper-paramètres est au cœur de la pratique du Deep Learning. La sélection judicieuse d'un optimiseur, d'un taux d'apprentissage approprié, combinée à une régularisation efficace constitue la base de tout entraînement réussi.

Du côté de l'architecture, les choix sur la profondeur du réseau, le type de couches, les fonctions d'activation et les mécanismes comme le pooling et le stride déterminent la capacité du modèle à capturer des patterns complexes dans les données.

Enfin, le Transfer Learning via des architectures robustes telles que ResNet et VGG représente une approche incontournable dans les projets réels, permettant d'obtenir d'excellentes performances même avec des données limitées.